NMC week 4

Exercise 1

For this exercise I was not able to get the loihi chip running on the model I selected in week 3. I get a weird notImplementedError, which I was not able to solve. Instead, I chose the squaring the input model:

```
import numpy as np
import matplotlib.pyplot as plt
!pip install nengo
!pip install nengo_loihi
import nengo
from nengo.processes import WhiteSignal
from nengo.utils.matplotlib import rasterplot
import nengo_loihi
import timeit

# All NengoLoihi models should call this before model construction
nengo_loihi.set_defaults()
```

```
import matplotlib.pyplot as plt
import numpy as np

import nengo

# Create the model object
model = nengo.Network(label="Squaring")
with model:
    # Create two ensembles of 100 leaky-integrate-and-fire neurons
    A = nengo.Ensemble(100, dimensions=1)
    B = nengo.Ensemble(100, dimensions=1)

     # Create an input node that represents a sine wave
    sin = nengo.Node(np.sin)

    # Connect the input node to ensemble A
    nengo.Connection(sin, A)

    # Define the squaring function
    def square(x):
        return x[0] * x[0]

    # Connection ensemble A to ensemble B
    nengo.Connection(A, B, function=square)

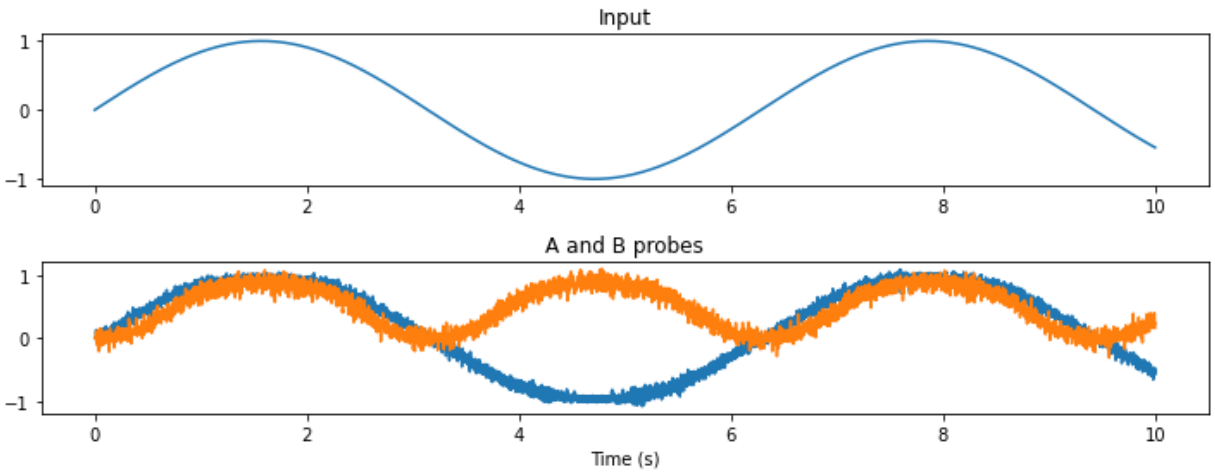
    sin_probe = nengo.Probe(sin)
    A_probe = nengo.Probe(A, synapse=0.01)
    B_probe = nengo.Probe(B, synapse=0.01)

# Create the simulator with nengo_loihi
start = timeit.default_timer()
with nengo_loihi.Simulator(model) as sim:
    sim.run(10)
    end = timeit.default_timer()
print(end - start)
```

```
13.771067631999813
```

```
def plot_decoded(t, data):
    plt.figure(figsize=(10, 4))
    plt.subplot(2, 1, 1)
    plt.title("Input")
    plt.plot(t, data[sin_probe])
    plt.subplot(2, 1, 2)
    plt.plot(t, data[A_probe])
    plt.subplot(2, 1, 2)
    plt.title("A and B probes")
    plt.plot(t, data[B_probe])
    plt.xlabel("Time (s)")
    plt.tight_layout()
```

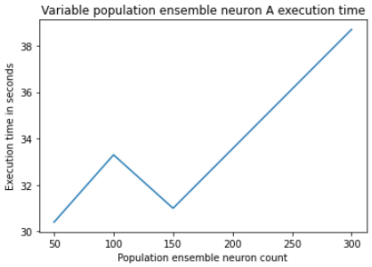
```
t = sim.trange()
plot_decoded(t, sim.data)
```



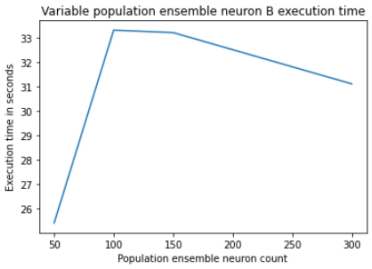
Variable neurons in the A and B ensembles will be changed and benchmarked

1. Manually filling in the time it took for the simulator to finish gives us the data:

```
prepop = [50, 100, 150, 300]
prepop_time = [30.4 , 33.3, 31.0, 38.7]
plt.plot(prepop, prepop_time)
plt.xlabel("Population ensemble neuron count")
plt.ylabel("Execution time in seconds")
plt.title("Variable population ensemble neuron A execution time")
plt.show()
```




```
prepop = [50, 100, 150, 300]
prepop_time = [25.4, 33.3, 33.2, 31.1]
plt.plot(prepop, prepop_time)
plt.xlabel("Population ensemble neuron count")
plt.ylabel("Execution time in seconds")
plt.title("Variable population ensemble neuron B execution time")
plt.show()
```



There is significant improvement on the Loihi chip, as all simulations seem to be faster on this chip than previous week. The longest real-time it took for the simulation to run is still quicker than the run-time it was allocated, which is very different.

2. We can only see a deterioration of results when the population of ensemble neuron A is increased. Neuron counts under 200 seem to have around the same simulation time, but higher counts really increases the time.

3. Variable population count in neuron A seem to cause deteriorated results, probably because computation increases exponentially when adding neurons to the population. This increases simulation time, I believe.
   Variable population count in neuron B does not seem to deteriorate when increasing the number. Probably because less connections are made between neurons in the group.

Exercise 2

1. PES: Prescribed Error Sensitivity learning rule, which modifies a connection's decoder to minimize an error signal provided through a connection to the connection's learning rule. Prescribed Error Sensitivity (PES) is a biologically plausible supervised learning rule that is frequently used with the Neural Engineering Framework (NEF). PES modifies the connection weights between populations of neurons to minimize an external error signal

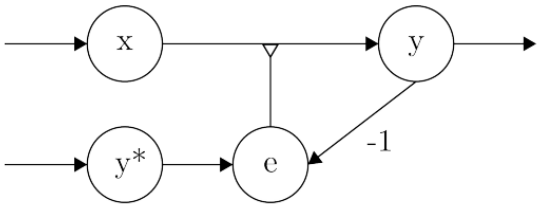


Figure 1: Network diagram used to analyze the PES rule. A connection from x to y learns to output y^* by minimizing $|y^* - y|$. The error signal forms a modulatory connection to modify the weights between x and y.

(Voelker, A. R. (2015). A solution to the dynamics of the prescribed error sensitivity learning rule. Centre for Theoretical N euro-science, Waterloo.)

2. From last graph below, we can see that the post decoding PES does not follow the actual product exactly. However, it does resemble the actual solution a lot. So, I would say it converges to the correct solution, but it is not perfect. From the first graph we can see that it converges around 35 seconds, as the red lines follow the black lines well.

3. No, it is not possible to get an exact solution, as there always seems to be some error. The input is noisy, but should not affect the output, as the noise is input. The output is affected by the synapses, as a delay is still there. This delay makes it harder for neurons get the exact solution, I believe.

```
from nengo.processes import WhiteSignal
nengo_loihi.set_defaults()
model = nengo.Network()
with model:
    #  input and pre popluation
    inp = nengo.Node(WhiteSignal(50, high=5), size_out=2)
    pre = nengo.Ensemble(100, dimensions=2)
    nengo.Connection(inp, pre)
    # reference population, containing the actual sum
    sum = nengo.Ensemble(50, dimensions=1)
    nengo.Connection(inp, sum, function=lambda x: x[0] + x[1], synapse=None)
    # post population
    post_pes = nengo.Ensemble(50, dimensions=1)
    # error population
    error_pes = nengo.Ensemble(50, dimensions=1)
    nengo.Connection(post_pes, error_pes)
    nengo.Connection(sum, error_pes, transform=-1)

    # learning rule
    conn_pes = nengo.Connection(
        pre,
        post_pes,
        function=lambda x: np.random.random(1),
        learning_rule_type=nengo.PES(),
    )
    nengo.Connection(error_pes, conn_pes.learning_rule)

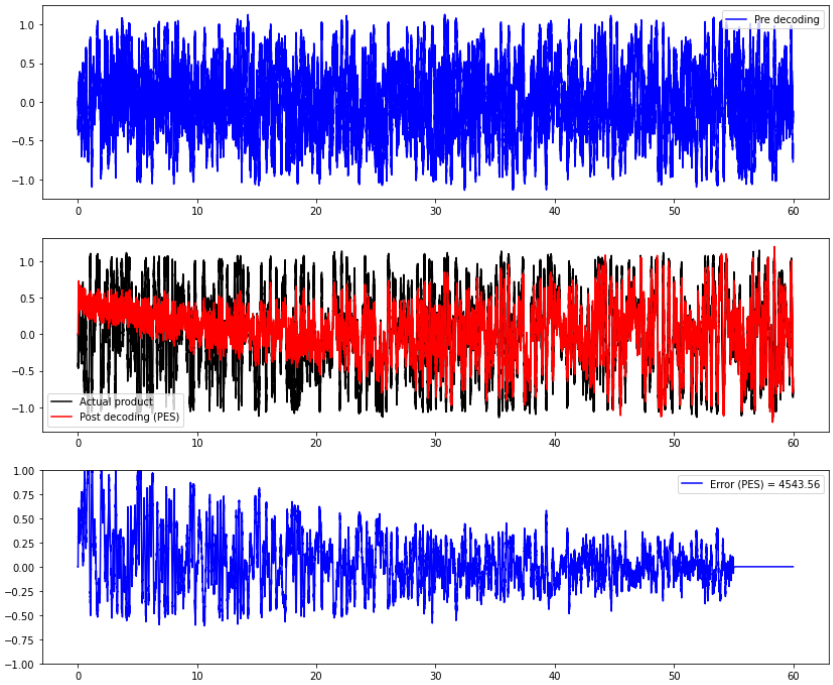
    # inhibit errors after 55 seconds
    inhib = nengo.Node(lambda t: 2.0 if t > 55.0 else 0.0)
    nengo.Connection(inhib, error_pes.neurons, transform=[[-1]] * error_pes.n_neurons)

    # probes
    sum_p = nengo.Probe(sum, synapse=0.01)
    pre_p = nengo.Probe(pre, synapse=0.01)
    post_pes_p = nengo.Probe(post_pes, synapse=0.01)
    error_pes_p = nengo.Probe(error_pes, synapse=0.03)

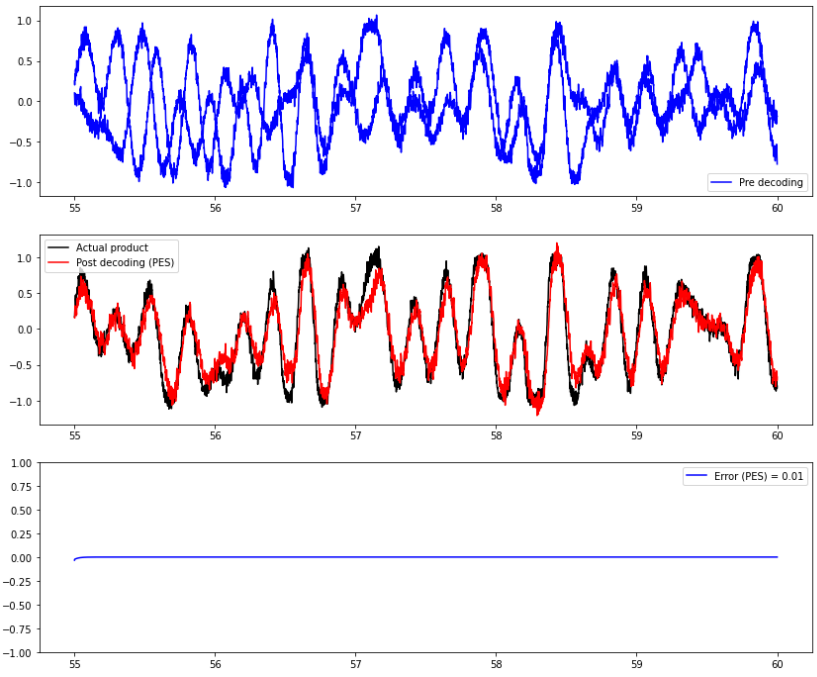
with nengo_loihi.Simulator(model) as sim:
    sim.run(60)
```

```
# from the nengo documentation
def plots(start_ix=None, end_ix=None):
    sl = slice(start_ix, end_ix)
    t = sim.trange()[sl]
    plt.figure(figsize=(14, 12))
    plt.suptitle("")
    plt.subplot(3, 1, 1)
    plt.plot(t, sim.data[pre_p][sl], c="b")
    plt.legend(("Pre decoding",), loc="best")
    plt.subplot(3, 1, 2)
    plt.plot(t, sim.data[sum_p][sl], c="k", label="Actual product")
    plt.plot(t, sim.data[post_pes_p][sl], c="r", label="Post decoding (PES)")
    plt.legend(loc="best")
    plt.subplot(3, 1, 3)
    ssqe = np.sum(np.square(sim.data[error_pes_p][sl]))
    plt.plot(t, sim.data[error_pes_p][sl], c="b", label="Error (PES) = {:.2f}".format(ssqe))
    plt.legend(loc="best")
    plt.ylim(-1, 1)
```

First graph:



Second graph:



Exercise 3

1. Oja modifies connection weights according to the Hebbian Oja rule, which augments typically Hebbian coactivity with a “forgetting” term that is proportional to the weight of the connection and the square of the postsynaptic activity. Over time the neuron actually learns to compute a principal component of its input stream. The change in a given synaptic weight is proportional to both the pre-synaptic input and the output activity of the post-synaptic neuron.

2. The Oja rule does not track the reference output well, compared to the PES learning rule. Oja is very good at tracking in the beginning of the simulation, but it diverges very fast (after 2 seconds, in the graph below). Compared to PES, Oja does not converge and error is very high. This is probably because of the forgetting term. As this is proportional to the weight of the connection, we can assume that the weight increases very fast in the beginning of the simulation. This probably means that the neurons learn very fast and the weight is high, which leads to a high forgetting term, basically resetting the weights and delivering very bad results because of it.

```
from nengo.processes import WhiteSignal

model = nengo.Network()
with model:
    inp = nengo.Node(WhiteSignal(100, high=5), size_out=2)
    pre = nengo.Ensemble(120, dimensions=2)
    post = nengo.Ensemble(100, dimensions=1)

    sum = nengo.Ensemble(50, dimensions=1)
    nengo.Connection(inp, sum, function=lambda x: x[0] + x[1], synapse=None)

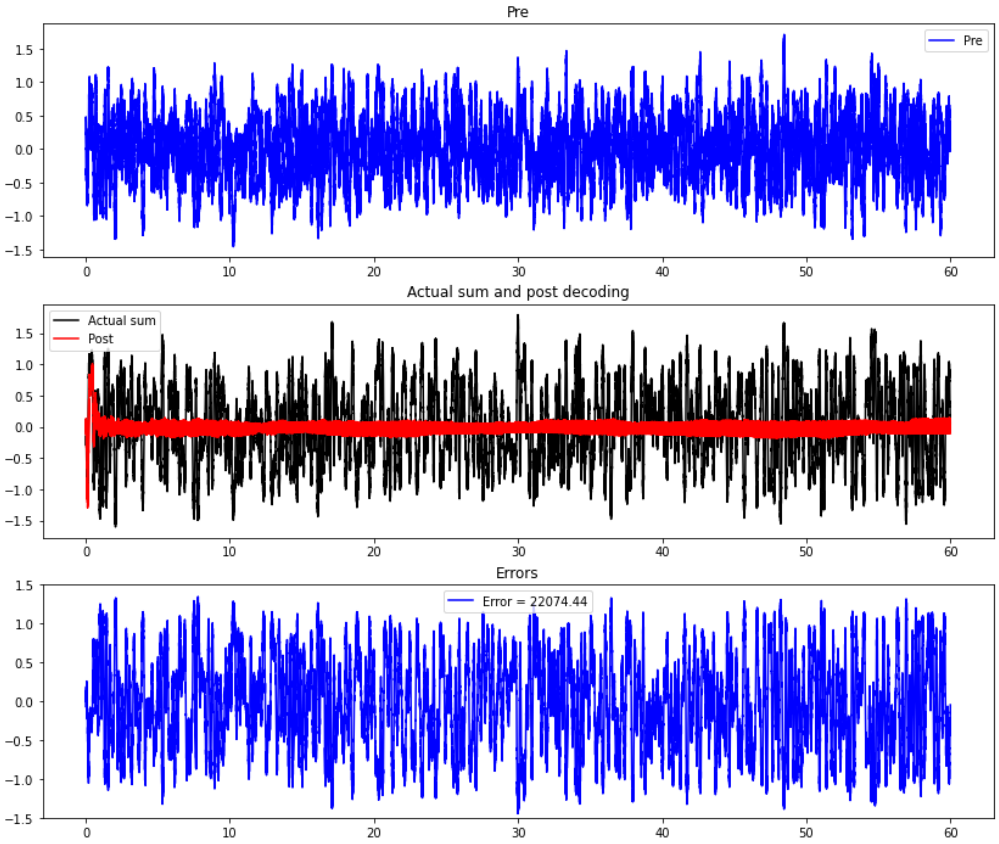
    nengo.Connection(inp, pre)
    conn = nengo.Connection(pre, post, function=lambda x: x[0] + x[1], solver=nengo.solvers.LstsqL2(weights=True))
    conn.learning_rule_type = nengo.Oja()

    error = nengo.Ensemble(60, dimensions=1)
    nengo.Connection(post, error)
    nengo.Connection(sum, error, transform=-1)

    pre_p = nengo.Probe(pre, synapse=0.01)
    post_p = nengo.Probe(post, synapse=0.01)
    sum_p = nengo.Probe(sum, synapse=0.01)
    error_p = nengo.Probe(error, synapse=0.01)

with nengo.Simulator(model) as sim:
    sim.run(60)
```

```
def plots(start_ix=None, end_ix=None):
    sl = slice(start_ix, end_ix)
    t = sim.trange()[sl]
    plt.figure(figsize=(14, 12))
    plt.subplot(3, 1, 1)
    plt.title("Pre")
    plt.plot(t, sim.data[pre_p][sl], c="b")
    plt.legend(("Pre",), loc="best")
    plt.subplot(3, 1, 2)
    plt.title("Actual sum and post decoding")
    plt.plot(t, sim.data[sum_p][sl], c="k", label="Actual sum")
    plt.plot(t, sim.data[post_p][sl], c="r", label="Post")
    plt.legend(loc="best")
    plt.subplot(3, 1, 3)
    plt.title("Errors")
    ssqe = np.sum(np.square(sim.data[error_p][sl]))
    plt.ylim(-1.5, 1.5)
    plt.plot(t, sim.data[error_p][sl], c="b", label="Error = {:.2f}".format(ssqe))
    plt.legend(loc="best")
```


Pre
Pre
Actual sum and post decoding
Actual sum
Post
Errors
Error = 22074.44